PROJECT REPORT

RoadDamage-XAI

Explainable Road Damage Classification Using Transfer Learning and Grad-CAM

Sharif MD Shahriar Tamjid
9 October 2026

Abstract

This report documents **RoadDamage-XAI**, a computer-vision project that classifies road damage into three categories — **Pothole**, **Crack** and **Manhole** — and explains each prediction visually using **Grad-CAM**. The source dataset is an object-detection dataset of 2,009 road images with 4,737 annotated instances in YOLO polygon format. Each annotated region was converted into an individual padded crop, producing 4,736 usable crops, and the data was split at the *original-image* level (70/15/15) so that crops from the same photograph never appear in more than one split.

A **MobileNetV2** network pretrained on ImageNet was adapted by (i) training a new classification head on a frozen backbone and (ii) fine-tuning the upper backbone layers with a much smaller learning rate. On the 692 held-out test crops the final model reached **69.51% accuracy** and a **63.33% macro F1-score**. Crack was recognised far better (F1 81.08%) than Pothole (56.23%) or Manhole (52.67%). Grad-CAM heatmaps from the final convolutional layer were used qualitatively to inspect which image regions influenced predictions, including incorrect ones.

Keywords: road damage, image classification, transfer learning, MobileNetV2, Grad-CAM, explainable AI.

Contents

1 Introduction and Purpose · 2 Dataset · 3 Methodology · 4 Models and Techniques · 5 Training · 6 Results · 7 Explainability with Grad-CAM · 8 Final Prediction Pipeline · 9 Discussion and Limitations · 10 Future Work · 11 Conclusion · Appendix A: Reproducibility and Outputs · References

1 Introduction and Purpose

1.1 Background and problem

Road-condition monitoring is an important computer-vision problem: inspecting large numbers of road images by hand is slow and hard to scale. A classifier can label damage types automatically, but a bare prediction does not show *why* the model decided as it did. That makes a model harder to analyse and to trust, particularly when it is wrong.

1.2 Purpose of the project

The project is built around two questions, stated in the project README:

- **What type of road damage is present?** — a three-class classification task (Pothole, Crack, Manhole).
- **Which part of the image influenced the model's decision?** — answered with Grad-CAM heatmaps.

To answer them the project develops a lightweight pipeline that (1) extracts annotated road-damage regions from YOLO polygon labels, (2) classifies each region, (3) reports a confidence score, and (4) overlays a Grad-CAM heatmap on the image. It therefore joins **computer vision, transfer learning, model evaluation and explainable AI** in one workflow.

1.3 Scope

The original dataset is designed for object *detection*. Here, the annotated regions are cut out and used to train an image *classifier*. The system consequently operates on **cropped damage regions**; it does not take a new full road scene and locate and classify every damage instance. That would need an additional detection or segmentation stage (see Section 10). The emphasis is on a complete, understandable experimental pipeline, not on claiming production-level accuracy.

2 Dataset

The project uses the Kaggle dataset *Road Damage Dataset: Potholes, Cracks and Manholes* (`lorenzoarcioni/road-damage-dataset-potholes-cracks-and-manholes`), downloaded in the notebook with `kagglehub`. The notebook locates one `images` folder and two label folders (`labels` and `labels-YOLO`); the `labels` folder is the one used. It contains 2,009 images and 2,009 label files.

Property	Value
Original images	2,009
Annotated objects	4,737
Extracted crops	4,736
Number of classes	3 (Pothole = 0, Crack = 1, Manhole = 2)
Model input size	224 × 224 × 3
Training / validation / test source images	1,406 / 301 / 302
Training / validation / test crops	3,303 / 741 / 692

Table 1 — Dataset summary (from the notebook outputs).

Each label line has the form `class_id x1 y1 x2 y2 …`, with polygon coordinates normalised to [0, 1]. The annotated instances are imbalanced: Crack accounts for roughly half of all objects.

Class	Annotated instances	Extracted crops
Pothole	1,261	1,261
Crack	2,519	2,518
Manhole	957	957
Total	4,737	4,736

Table 2 — Class distribution. One Crack annotation did not yield a usable crop.

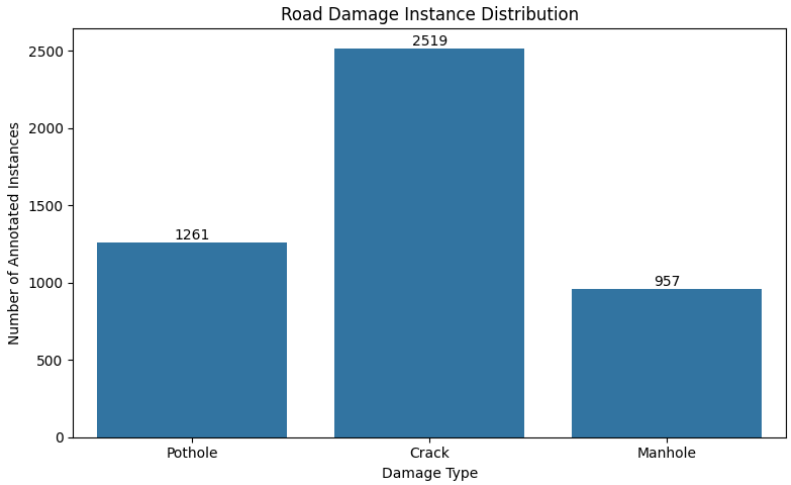


Figure 1 — Distribution of annotated instances per class (notebook output).

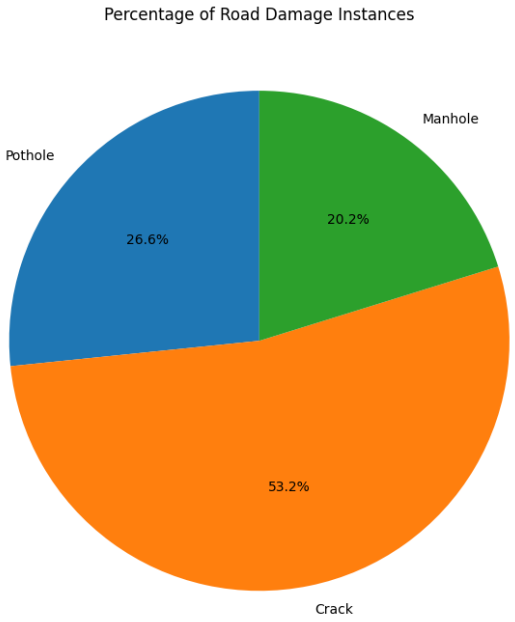


Figure 2 — Percentage of annotated instances per class (notebook output).



Figure 3 — Six randomly selected images with their polygon annotations drawn in red (notebook output).

3 Methodology

The complete pipeline, as set out in the notebook and README, is shown below.

1. YOLO polygon annotations	**2.** Annotation parsing	**3.** Polygon → padded bounding box
4. Damage-crop extraction	**5.** Image-level train / validation / test split	**6.** Preprocessing + augmentation
7. MobileNetV2 transfer learning	**8.** Fine-tuning	**9.** Evaluation → Grad-CAM → prediction, confidence and heatmap

Figure 4 — End-to-end pipeline.

3.1 Parsing the annotations

A parser reads every YOLO label file, skips lines with fewer than nine values (i.e. fewer than four polygon points), and reshapes the remaining coordinates into (x, y) pairs. Each label file is matched to the image of the same name. The result is a table of 4,737 annotated objects holding the image path, label path, class and polygon.

3.2 Extracting damage crops

For every object the polygon is reduced to its axis-aligned bounding box. The box is expanded by **15% padding** on each side (clipped to the image borders) to preserve some surrounding road context, and the region is saved as a JPEG crop in a per-class folder. One annotation produced an empty crop and was skipped, leaving **4,736 crops**: Pothole 1,261, Crack 2,518, Manhole 957.

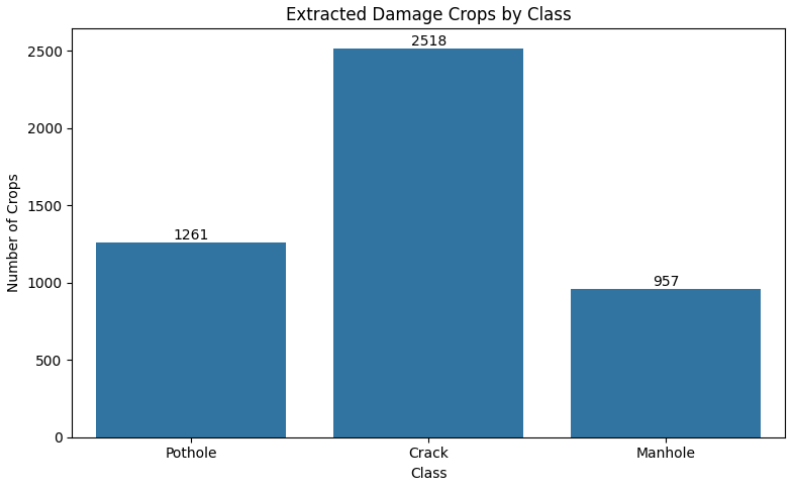


Figure 5 — Number of extracted crops per class (notebook output).



Figure 6 — Five random example crops per class (rows: Pothole, Crack, Manhole). Crops vary widely in size, aspect ratio and resolution (notebook output).

3.3 Image-level data splitting

The split is performed on the **original source images**, not on the extracted crops. This prevents crops of the same photograph from landing in both training and test sets, which would make the evaluation optimistic. Source images were split 70% / 15% / 15% with `train_test_split` (random seed 42), stratified by a per-image label. For images that contain several classes, the first encountered class is used for stratification only. Crops were then assigned to the split of their source image:

Split	Source images	Crops
Training	1,406	3,303
Validation	301	741
Test	302	692

Table 3 — Result of the image-level split.

3.4 Preprocessing and augmentation

A `tf.data` pipeline reads each crop, decodes it as 3-channel JPEG, resizes it to **224 × 224**, casts it to float and applies the MobileNetV2 `preprocess_input` function. The training set is shuffled (seed 42), and all sets are batched (**batch size 32**) and prefetched. Augmentation is a Keras `Sequential` block built into the front of the model, so it acts only during training:

- Random horizontal flip
- Random rotation (factor 0.08)
- Random zoom (factor 0.10)
- Random contrast (factor 0.10)

The aim is to make the classifier less dependent on the exact orientation and appearance of the training images.

4 Models and Techniques

4.1 Transfer learning

Training a deep CNN from scratch needs a great deal of labelled data. **Transfer learning** instead starts from a network already trained on a large dataset (here ImageNet) and reuses its learned visual features. Only a small new classification head is trained first; deeper layers can then be adapted gently to the new domain. This project uses exactly that two-stage recipe.

4.2 MobileNetV2 (feature extractor)

MobileNetV2 (Sandler et al., 2018) is a compact convolutional network designed for efficient inference. Its main building blocks are depthwise-separable convolutions, which factorise a standard convolution into a cheap per-channel spatial filter and a 1×1 channel-mixing convolution, and *inverted residual blocks with linear bottlenecks*, which expand a narrow representation, filter it, and project it back down with a residual connection. This keeps the parameter count and compute low. In this project the backbone is loaded with ImageNet weights and without its original classification top (`include_top=False`). For a 224 × 224 input it outputs a 7 × 7 × 1280 feature map from its last convolutional layer, named `Conv_1`. The backbone has 2,257,984 parameters.

4.3 Classification model

Layer	Output shape	Parameters
input_image (Input)	(None, 224, 224, 3)	0
data_augmentation (Sequential)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling	(None, 1280)	0
dropout (rate 0.30)	(None, 1280)	0
predictions (Dense, softmax)	(None, 3)	3,843
Total		**2,261,827** (8.63 MB)

Table 4 — Model architecture (output of `model.summary()`). At the start of head training, 3,843 parameters are trainable and 2,257,984 are frozen.

Global average pooling collapses each of the 1,280 feature maps to one number, **dropout** randomly disables 30% of those features during training to reduce over-fitting, and the final **dense softmax layer** turns them into probabilities for the three classes. The model has approximately

2.26 million parameters.

4.4 Grad-CAM (explainability)

Gradient-weighted Class Activation Mapping (Grad-CAM; Selvaraju et al., 2017) explains a CNN's decision for a chosen class. The gradient of the class score with respect to the feature maps of a convolutional layer is averaged over the spatial dimensions to give one importance weight per channel; the feature maps are combined using these weights, and a ReLU keeps only regions with a *positive* influence on the class. The result is a coarse heatmap that is overlaid on the image.

The notebook implements this by hand. It builds a sub-model of the MobileNetV2 backbone that outputs both the `Conv_1` activations and the final backbone feature map, then re-applies global average pooling and the trained `predictions` layer inside a `GradientTape`. (Rebuilding the pooled-and-dense part manually avoids the Keras nested-model graph problem noted in the notebook's comments.) The heatmap is the ReLU of the gradient-weighted sum of the feature maps, normalised to [0, 1], giving a 7 × 7 map. Because the dense layer ends in softmax, the class score being differentiated is the softmax probability of the target class.

5 Training

5.1 Configuration

Setting	Stage 1: head training	Stage 2: fine-tuning
Backbone	Fully frozen	First 100 backbone layers frozen; the rest trainable
Optimizer	Adam	Adam
Learning rate	1e-3	1e-5
Loss	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Metric	Accuracy	Accuracy
Max epochs	10	10
Batch size	32	32
Epochs actually run	8 (early stopping)	9 (early stopping)

Table 5 — Training set-up.

Two callbacks, shared by both stages, are used: **EarlyStopping** on validation loss (patience 4, best weights restored) and **ReduceLROnPlateau** on validation loss (factor 0.2, patience 2, minimum learning rate 1e-7). The learning rate during head training fell from 1e-3 to 2e-4 (epoch 4) and 4e-5 (epoch 7); during fine-tuning it fell from 1e-5 to 2e-6 (epoch 8). After fine-tuning, the model had **1,865,283 trainable parameters** out of 2,261,827. The notebook reports no GPU (an empty device list), and epoch times were roughly 115–145 s for head training and 170–206 s for fine-tuning.

5.2 Epoch-by-epoch results

Stage	Epoch	Train acc.	Train loss	Val. acc.	Val. loss
Head	1	0.5123	1.0441	0.6437	0.7938
Head	2	0.5640	0.9095	0.6370	0.8244
Head	3	0.5771	0.8652	0.6248	0.9264

Stage	Epoch	Train acc.	Train loss	Val. acc.	Val. loss
Head	4	0.5943	0.8510	0.6775	0.7122
Head	5	0.6119	0.8217	0.6788	0.7555
Head	6	0.5949	0.8218	0.6640	0.7313
Head	7	0.6110	0.8046	0.6694	0.7277
Head	8	0.6070	0.8199	0.6653	0.7327
Fine-tune	1	0.5540	0.9470	0.6748	0.7047
Fine-tune	2	0.5749	0.8895	0.6964	0.6936
Fine-tune	3	0.5889	0.8535	0.6950	0.6919
Fine-tune	4	0.5861	0.8451	0.6964	0.6855
Fine-tune	5	0.6016	0.8143	0.7018	0.6760
Fine-tune	6	0.6043	0.8121	0.7018	0.6877
Fine-tune	7	0.6188	0.7966	0.7031	0.7074
Fine-tune	8	0.6291	0.7914	0.7004	0.6968
Fine-tune	9	0.6364	0.7705	0.6991	0.6969

Table 6 — Per-epoch metrics logged by the notebook. The lowest validation loss of the whole run (0.6760) occurs at fine-tuning epoch 5.

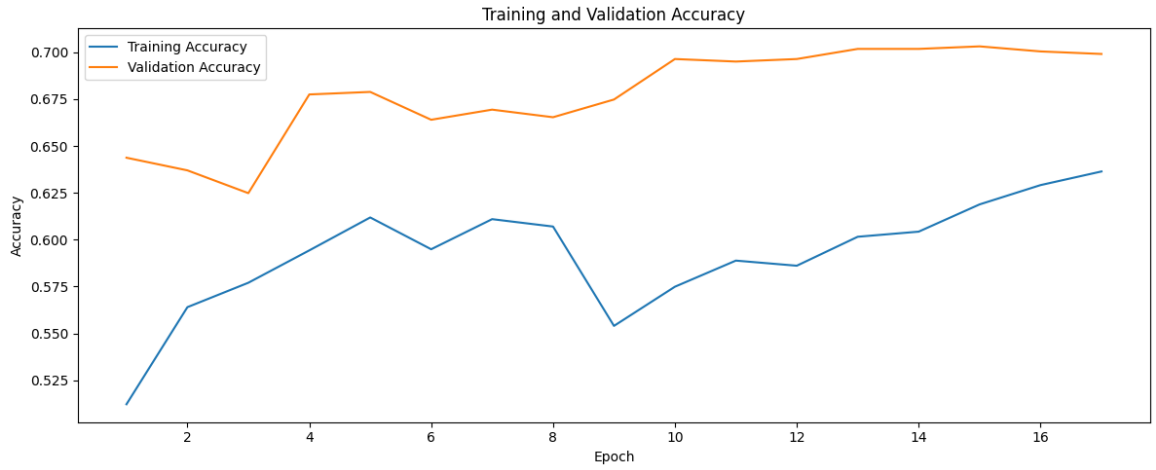


Figure 7 — Training and validation accuracy across both stages (17 epochs in total; fine-tuning begins at epoch 9) (notebook output).



Figure 8 — Training and validation loss across both stages (notebook output).

Validation accuracy rises from 64.4% (first epoch) to about 70% by the end of fine-tuning. Training accuracy is lower than validation accuracy throughout; this is plausible because augmentation and dropout are active only when training, but the notebook does not test this explanation. Training accuracy dips at the first fine-tuning epoch (0.6070 → 0.5540) as the unfrozen layers begin to adapt, and then climbs steadily.

6 Results

The final model was evaluated once on the **692 unseen test crops**. Test loss was 0.7159 and test accuracy 0.6951.

6.1 Overall performance

Metric	Score
Accuracy	69.51%
Macro precision	63.85%
Macro recall	62.91%
Macro F1	63.33%

Table 7 — Overall test-set metrics. The model made 481 correct and 211 incorrect predictions. Macro metrics average the per-class scores equally, which matters because the classes are imbalanced.

6.2 Per-class performance

Class	Precision	Recall	F1-score	Support
Pothole	58.08%	54.49%	56.23%	178
Crack	**79.15%**	**83.11%**	**81.08%**	379
Manhole	54.33%	51.11%	52.67%	135
Weighted average	68.89%	69.51%	69.15%	692

Table 8 — Per-class test metrics (scikit-learn classification report).

6.3 Confusion matrices

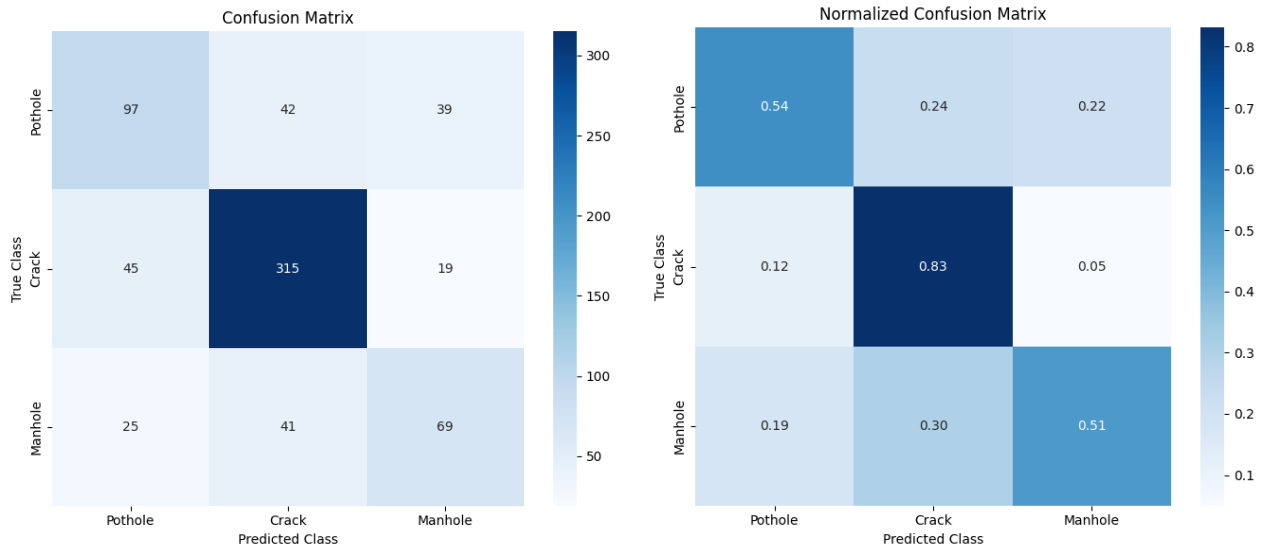


Figure 9 — Confusion matrix (left, counts) and row-normalised confusion matrix (right) on the test set (notebook output).

True \ Predicted	Pothole	Crack	Manhole	Total
Pothole	97	42	39	178
Crack	45	315	19	379
Manhole	25	41	69	135

Table 9 — Confusion-matrix counts, read from the matrix in Figure 9 (the diagonal sums to 481, matching the notebook's count of correct predictions).

6.4 Interpretation

- **Crack is the easiest class.** 315 of 379 cracks (83%) were correct; F1 is 81.08%. Crack's visual characteristics are more consistently captured by the learned features.
- **Pothole and Manhole are harder.** Recall is only 54% and 51%. Potholes were misread as Crack (24%) or Manhole (22%); Manhole was misread as Crack most often (30%) and as Pothole (19%).
- **Accuracy flatters the model.** Accuracy (69.51%) is above macro F1 (63.33%) because strong Crack performance on the largest class contributes heavily to overall accuracy.
- **Class imbalance matters.** Crack is more common than Pothole and Manhole in the data. The notebook does not apply class weighting or resampling.

Overall, MobileNetV2 provides a practical, compact baseline: close to 70% test accuracy on this three-class problem without training a large CNN from scratch.

7 Explainability with Grad-CAM

Grad-CAM was applied to the `Conv_1` layer of MobileNetV2, targeting the model's predicted class. In the figures below, warm colours (red/yellow) mark regions that most strongly increased the predicted class's score and cool colours (blue) mark regions with little positive influence.

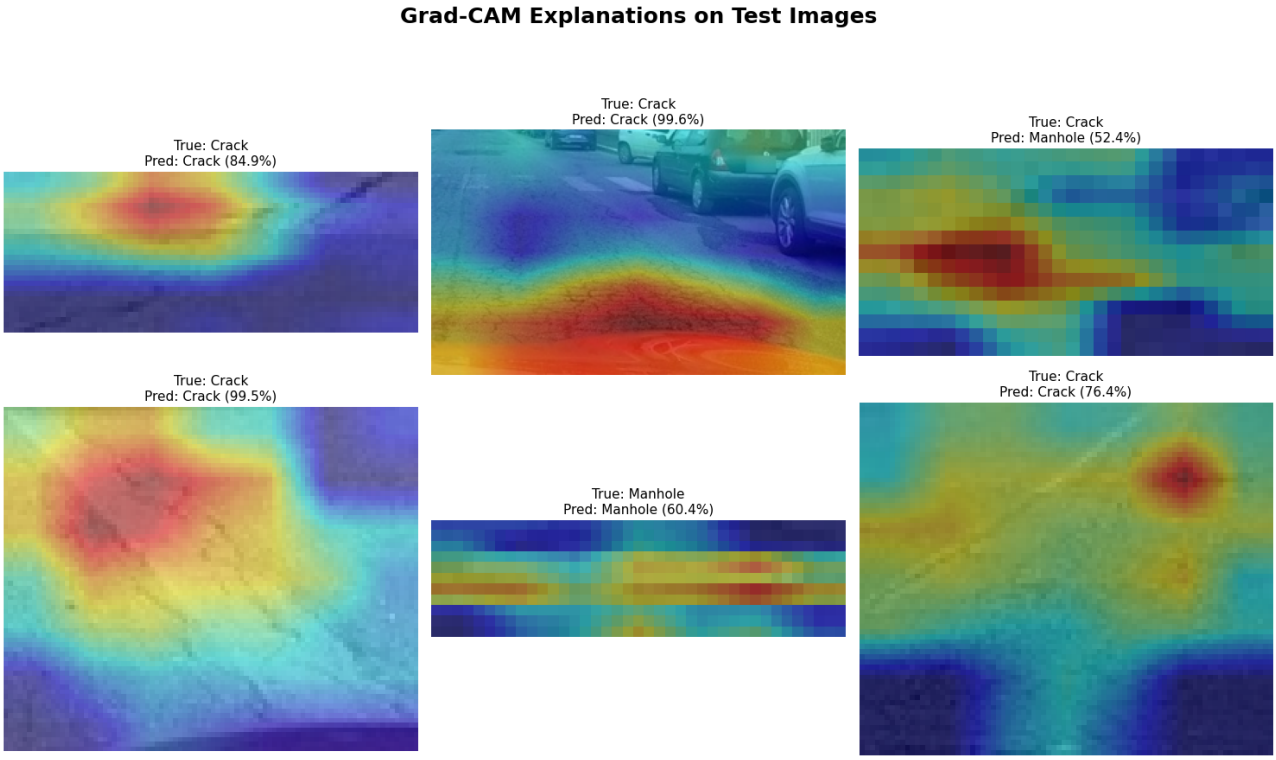


Figure 10 — Grad-CAM overlays for six randomly chosen test crops, with true class, predicted class and confidence (notebook output).

Of these six examples, five are classified correctly (confidences between 60.4% and 99.6%) and one is wrong: a true Crack predicted as Manhole with 52.4% confidence. In several of the Crack examples the strongest activation sits on the cracked road surface itself. Because the map is only 7 × 7 and is up-sampled to the crop, the highlighted areas are coarse and blocky, and they should be read as indicating *approximate* regions of influence, not precise damage boundaries.

7.1 Misclassified examples

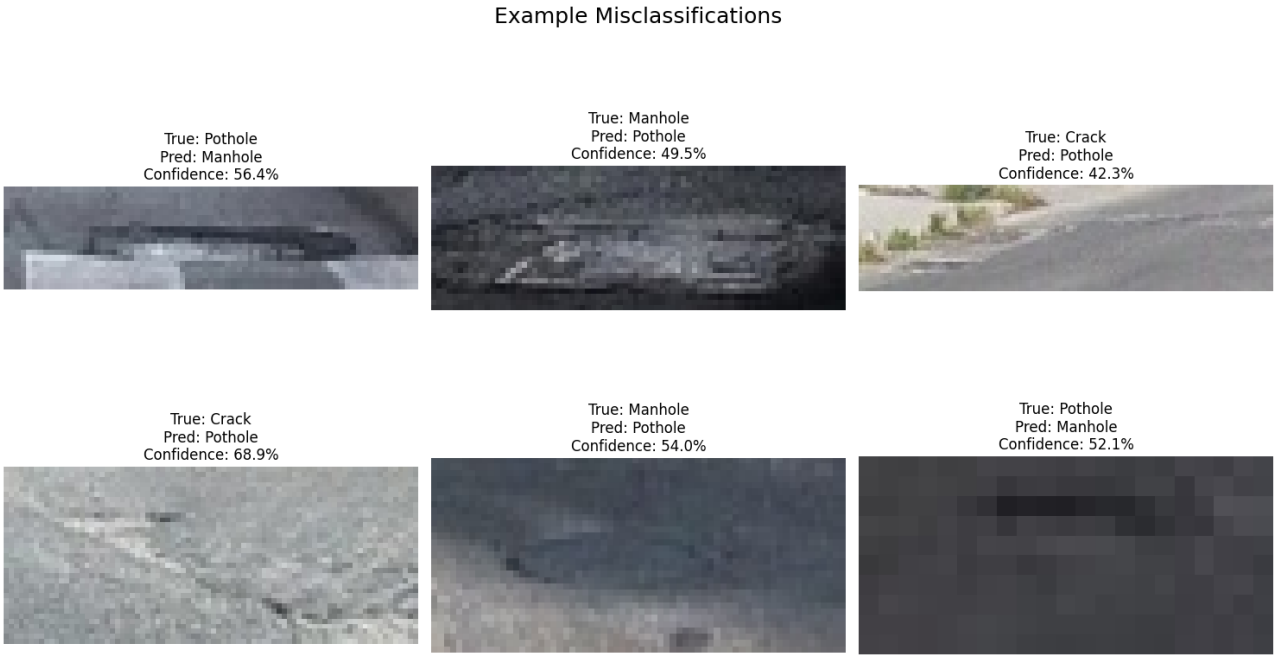


Figure 11 — Six randomly sampled misclassifications from the 211 test errors, with true class, predicted class and confidence (notebook output).

The sampled errors have confidences of only 42.3%–68.9%, i.e. the model was not strongly committed to its wrong answers. Visually, several of these crops are small and low-resolution (pixelation is clearly visible), and some show little obvious structure, which makes the class hard to determine even by eye. This is an observation from the six samples shown; the notebook does not perform a systematic error analysis.

Note on scope of the explanations. Grad-CAM is used qualitatively in this project. No separate quantitative evaluation of Grad-CAM faithfulness (for example, agreement between highlighted regions and the annotated damage areas) is performed.

8 Final Prediction Pipeline

The notebook packages the whole workflow into a single function, `predict_road_damage`. Given a crop path it reads the image, resizes it to 224 × 224, applies MobileNetV2 preprocessing, obtains class probabilities, selects the arg-max class, computes the Grad-CAM heatmap for that class and returns the prediction, confidence, probability vector, heatmap, overlay and original image.

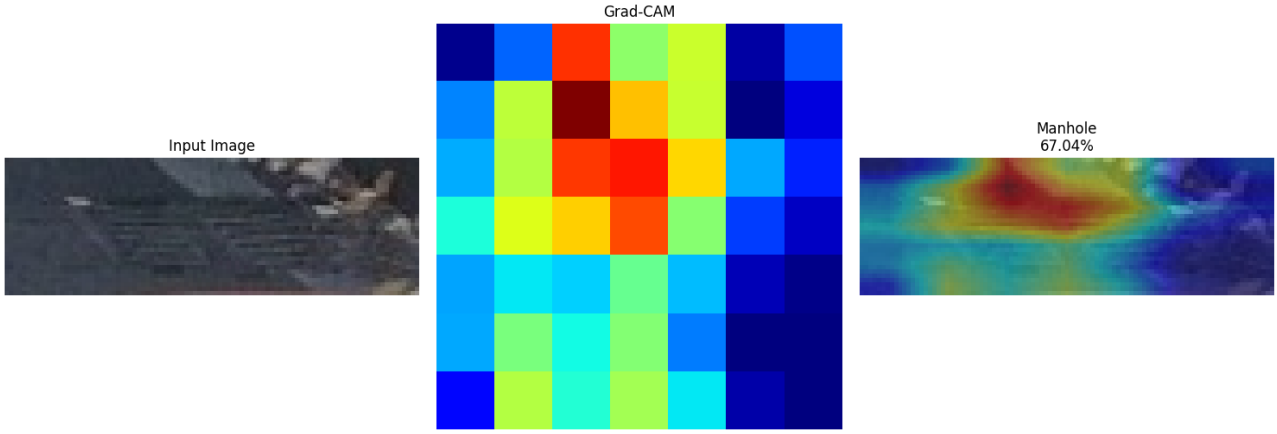


Figure 12 — Demonstration on one test crop: input image, Grad-CAM heatmap, and overlay with prediction "Manhole", confidence 67.04% (notebook output).

The 67.04% confidence applies to this single demonstration image and is not the model's overall confidence.

9 Discussion and Limitations

9.1 Key findings

- The leakage-aware, image-level split gives a more trustworthy estimate than a random crop-level split.
- Pretrained MobileNetV2 features transfer to road-damage crops, reaching 69.51% accuracy / 63.33% macro F1 with a small model (about 2.26 M parameters).
- Per-class metrics reveal what accuracy hides: Crack is strong, Pothole and Manhole remain weak.
- Grad-CAM adds an inspectable layer of evidence to every prediction, including the wrong ones.

9.2 Limitations

- **Classification, not detection.** The model labels pre-cropped regions supplied by ground-truth annotations; it cannot yet process a full road scene.
- **Moderate absolute performance.** Pothole and Manhole F1-scores are near 55%. The project is a baseline, not a production system.

- **Class imbalance is not addressed** (no class weights, resampling or targeted augmentation).
- **Aspect-ratio distortion.** Crops of varied shapes are resized directly to 224 × 224 without preserving aspect ratio, and many crops are small and low-resolution. The notebook does not test how much either affects accuracy.
- **Padding with context.** The 15% padding includes some surrounding road, which may add useful context or noise; this choice was not ablated.
- **Single run.** One seed (42) and one train/validation/test split were used; no confidence intervals or repeated runs are reported.
- **Stratification approximation.** Multi-class images were stratified by their first encountered class only.
- **Qualitative explanations only.** Grad-CAM faithfulness was not measured.
- **Robustness untested.** No experiments on lighting, viewpoint, weather, surface type or image quality.

10 Future Work

- **End-to-end detection:** combine localisation and classification so that a whole road image is processed directly, with explainable predictions.
- **Better minority-class performance:** more balanced data, class weighting, targeted augmentation and additional Pothole/Manhole samples.
- **Compare architectures:** for example EfficientNet, ResNet or newer lightweight vision models, studying accuracy, size and inference cost.
- **Stronger explainability:** compare Grad-CAM with other XAI methods and evaluate quantitatively whether highlights align with annotated damage.
- **Segmentation:** keep the polygon information rather than reducing it to a bounding box, giving more precise localisation of irregular damage such as cracks.
- **Real-world robustness:** test under different lighting, viewpoints, weather, road surfaces and image quality.

11 Conclusion

RoadDamage-XAI demonstrates an end-to-end workflow for an explainable computer-vision classifier for road-condition analysis. It converts YOLO polygon annotations into damage crops, uses an image-level split to reduce leakage, applies MobileNetV2 transfer learning and fine-tuning, evaluates with overall and class-wise metrics, and uses Grad-CAM to make predictions easier to interpret. The final model achieves **69.51% test accuracy** and a **63.33% macro F1-score**, with Crack substantially easier to classify than Pothole or Manhole. Explainability is treated as part of the machine-learning workflow rather than an afterthought, providing a practical foundation for more robust, fully automated road-damage detection.

Appendix A Reproducibility and Outputs

Item	Detail
Language / framework	Python; TensorFlow 2.20.0 / Keras
Libraries	OpenCV, NumPy, Pandas, scikit-learn, Matplotlib, Seaborn, kagglehub
Random seed	SEED = 42, applied to Python, NumPy and TensorFlow; also used for the data split and shuffling

Item	Detail
Hardware reported	No GPU detected in the notebook run
Saved model	road_damage_mobilenetv2.keras (trained MobileNetV2 model)
Saved metrics	road_damage_metrics.csv (accuracy, macro precision, macro recall, macro F1)
Saved predictions	test_predictions.csv (test-set predictions, confidence and correctness flag)
Project files	README.md and RoadDamage_XAI.ipynb; the model, CSVs and crops are generated in the notebook environment (/kaggle/working/), not stored in the project repository

Table 10 — Reproducibility details. Running the notebook top to bottom reproduces the data-processing, training, evaluation and Grad-CAM workflow.

Note on the exported notebook. The Grad-CAM cells call a helper named `overlay_gradcam`, which blends the heatmap with the image, but its definition is not present among the notebook cells provided for this report. Its output is visible in the figures, so it was evidently defined in the original session; this report describes only what the figures show.

References

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *IEEE International Conference on Computer Vision (ICCV)*.

Road Damage Dataset: Potholes, Cracks and Manholes. Kaggle dataset `lorenzoarcioni/road-damage-dataset-potholes-cracks-and-manholes`. Refer to the dataset page for licensing and attribution requirements.

RoadDamage-XAI project *README.md* and *RoadDamage_XAI.ipynb* (primary sources for this report).